Chapter 2: Decision Tree Tutorials

Decision trees is a form of supervised learning for classification and regression. The goal is to create a predictive model using decision rules inferred from the data features.

2.1 Decision Tree for Classification

In decision tree classifier, the target variable is a nominal variable.

Example 1: pima-indians-diabetes.xlsx (source: https://gist.github.com/ktisha/c21e73a1bd1700294ef790c56c8aec1f)

Column Names, Description and units for the dataset

Column No	Name	Description	Unit
1	pregNo	Number of times pregnant	
2	glucose	Plasma glucose concentration a 2 hours in an oral glucose tolerance test	
3	bp	Diastolic blood pressure	mm Hg
4	skin	Triceps skin fold thickness	mm
5	insulin	2-Hour serum insulin	mu U/ml
6	bmi	Body mass index (weight in kg/(height in m)^2)	Kg/m^2
7	pedigree	Diabetes pedigree function	
8	age	Age	years
9	class	Class variable (0 or 1)	

Make sure your pima-indians-diabetes.xlsx is in the same folder as your python or jupter notebook folder.

1. Type the following codes to read in the dataset and display the first 5 records of the dataset:

```
import pandas as pd

dataset = pd.read_excel('pima-indians-diabetes.xlsx')
dataset.head()
```

2. Type the following codes to provide descriptive statistics for all the quantitative variables in the dataset (except for the variable class):

```
import pandas as pd

dataset = pd.read_excel('pima-indians-diabetes.xlsx')
print(dataset.head())
print("\n")
print(dataset[['pregNo', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree', 'age']].describe())
```

3. We would need to download sklearn libraries. Split the dataset into features (independent variables) and target (dependent) variables. We shall choose all the variables as features except for Columns 4 and 9, because we are choosing Column 9 as the target variable. We shall chose test dataset as 30% (0.3) and thus training set is 70%. We shall use a decision tree classifier to build the decision tree model. We shall evaluate the accuracy of the model using the function metrics.accuracy_score() and confusion Matrix.

```
# Load libraries
import pandas as pd
from sklearn.tree import DecisionTreeClassifier # Import Decision Tree Classifier
from sklearn.model_selection import train_test_split # Import train_test_split
function
from sklearn import metrics #Import scikit-learn metrics module for accuracy
calculation
from sklearn.metrics import confusion_matrix

#Read in the dataset and display
dataset = pd.read_excel('pima-indians-diabetes.xlsx')
print(dataset.head())

#Feature selection: split the dataset into features (independent variables) and target
(dependent variable)
feature_cols = ['pregNo', 'glucose', 'bp', 'insulin','bmi','pedigree', 'age']
X = dataset[feature_cols] # Features
Y = dataset['class'] # Target variable

#Function for split dataset will have 3 parameters: features, target, test_set size
# Split dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3, random_state=1)
# 70% training and 30% test


#Build Decision Tree Model using Scikit Learn
# Create Decision Tree classifer object
clf = DecisionTreeClassifier()

# Train Decision Tree Classifer
clf = clf.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test)


#Evaluate the accuracy of the model (or classifier) for prediction
# Model Accuracy, how often is the classifier correct?
print("\n")
print("Accuracy for 70% training set and 30% test set :",
      metrics.accuracy_score(y_test, y_pred))
```

4. Evaluate the Model

    a. Accuracy % The output for (3) is :

```
Accuracy for 70% training set and 30% test set :
0.6753246753246753
```

The accuracy is 67.5%.

Try to build a model with the following number of features (with various combinations) and measure their accuracies:

- 6 features
- 5 features
- 4 features
- 3 features
- 2 features

b. Confusion Matrix – is also used so evaluate the accuracy of a classification.

In Scikit Learn

C[0,0] – True Negatives

C[1,0] - False Negatives

C[1,1] – True Positives

C[0,1] – False Positives

C[0,0] True Negatives	C[0,1] False Positives
C[1,0] False Negatives	C[1,1] True Positives

Types the following codes:

```
# Load libraries
import pandas as pd
from sklearn.tree import DecisionTreeClassifier # Import Decision
Tree Classifier
from sklearn.model_selection import train_test_split # Import
train_test_split function
from sklearn import metrics #Import scikit-learn metrics module for
accuracy calculation
from sklearn.metrics import confusion_matrix

#Read in the dataset and display
dataset = pd.read_excel('pima-indians-diabetes.xlsx')
print(dataset.head())

#Feature selection: split the dataset into features (independent
variables) and target (dependent variable)
feature_cols = ['pregNo', 'glucose', 'bp', 'insulin','bmi','pedigree',
'age']
```

```
X = dataset[feature_cols] # Features
Y = dataset['class'] # Target variable

#Function for split dataset will have 3 parameters: features, target,
test_set size
# Split dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=1) # 70% training and 30% test


#Build Decision Tree Model using Scikit Learn
# Create Decision Tree classifer object
clf = DecisionTreeClassifier()

# Train Decision Tree Classifer
clf = clf.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test)


#Evaluate the accuracy of the model (or classifier) for prediction
# Model Accuracy, how often is the classifier correct?
print("\n")
print("Accuracy for 70% training set and 30% test set :",
        metrics.accuracy_score(y_test, y_pred))


#How to improve the accuracy of the model? By tuning the number
of features for the model

#Confusion matrix
print("\n")
print(confusion_matrix(y_test, y_pred))
print("\n")

cm = confusion_matrix(y_test, y_pred)
tn = cm[0][0]
fn = cm[1][0]
tp = cm[1][1]
fp = cm[0][1]

print("true negative: ", tn)
print("false negative: ", fn)
print("true positive: ", tp)
print("false positive: ", fp)
```

The Output

```
Accuracy for 70% training set and 30% test set :
0.6753246753246753


[[113  33]
 [ 42  43]]


true negative:  113
false negative:  42
true positive:  43
false positive:  33
```

To verify accuracy = 67.5%
Use the formula:
(TN + TP)/(TN + FN + FP + TP) = 156/231 x 100% = 67.5%

Other measures of accuracy are: ROC and Precision-Recall curves; AUC, etc…

Resource
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

c. **Plot the Decision Tree**

Use text

Type the following (to load the iris.csv dataset):

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
iris = load_iris()
decision_tree = DecisionTreeClassifier(random_state=0, max_depth=2)
decision_tree = decision_tree.fit(iris.data, iris.target)
r = export_text(decision_tree, feature_names=iris['feature_names'])
print(r)
```

Output is as follows:

```
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2
```

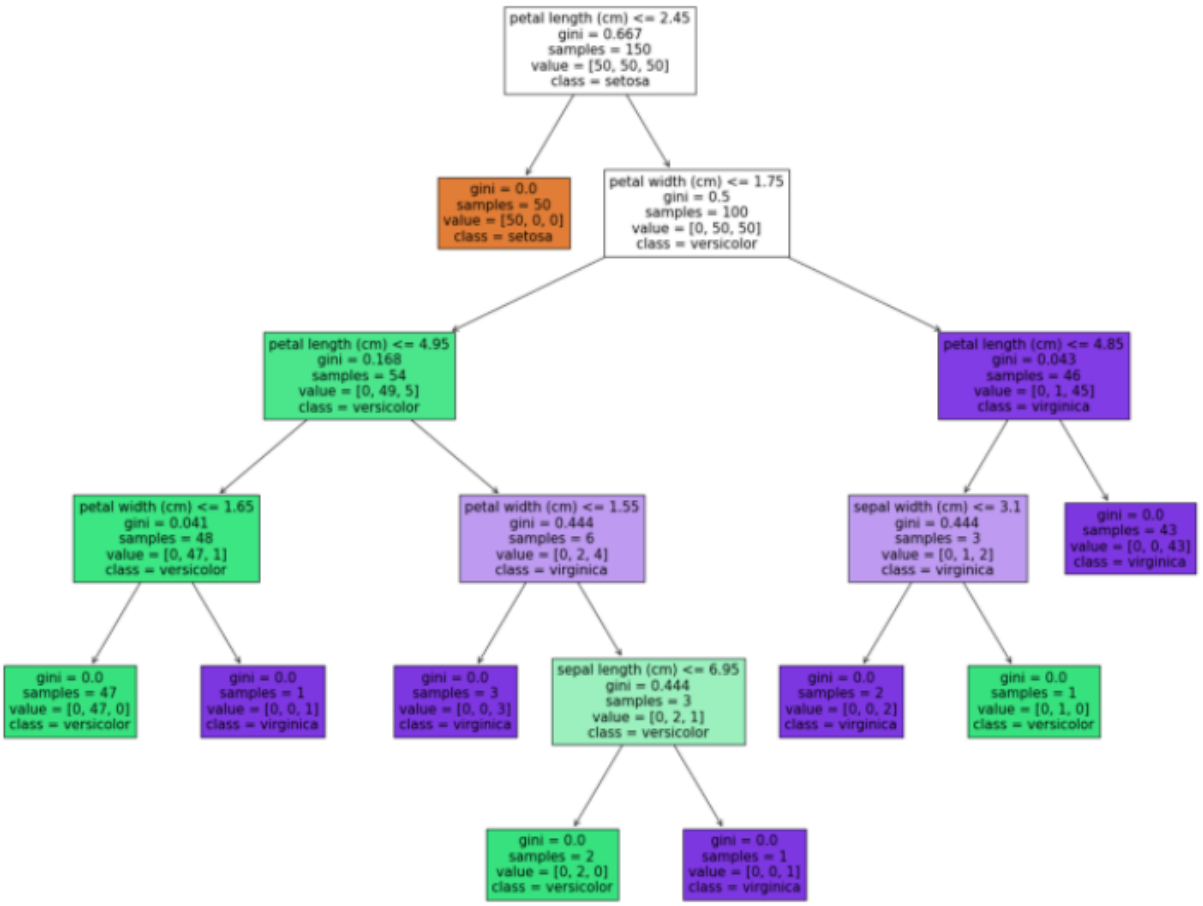
Use Text and Tree

Type the following codes:

```
#https://mljar.com/blog/visualize-decision-tree/
from matplotlib import pyplot as plt
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

# Prepare the data data
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Fit the classifier with default hyper-parameters
clf = DecisionTreeClassifier(random_state=1234)
model = clf.fit(X, y)

text_representation = tree.export_text(clf)
print(text_representation)

fig = plt.figure(figsize=(25,20))
tree.plot_tree(clf,
               feature_names=iris.feature_names,
               class_names=iris.target_names,
               filled=True)
```

Output is as follows:

```
|--- feature_2 <= 2.45
|   |--- class: 0
|--- feature_2 >  2.45
|   |--- feature_3 <= 1.75
|   |   |--- feature_2 <= 4.95
|   |   |   |--- feature_3 <= 1.65
|   |   |   |   |--- class: 1
|   |   |   |--- feature_3 >  1.65
|   |   |   |   |--- class: 2
|   |   |--- feature_2 >  4.95
|   |   |   |--- feature_3 <= 1.55
|   |   |   |   |--- class: 2
|   |   |   |--- feature_3 >  1.55
|   |   |   |   |--- feature_0 <= 6.95
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- feature_0 >  6.95
|   |   |   |   |   |--- class: 2
|   |--- feature_3 >  1.75
|   |   |--- feature_2 <= 4.85
|   |   |   |--- feature_1 <= 3.10
|   |   |   |   |--- class: 2
|   |   |   |--- feature_1 >  3.10
|   |   |   |   |--- class: 1
|   |   |--- feature_2 >  4.85
|   |   |   |--- class: 2
```



d. Resources

https://scikit-learn.org/stable/modules/tree.html

https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html

https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html#sphx-glr-download-auto-examples-model-selection-plot-confusion-matrix-py

Resources
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html
https://scikit-learn.org/stable/modules/tree.html

2.2 Decision Tree for Regression

Regression trees have target variables that are numeric or continuous.

Example 2.2.1 – Direct data

Type the following to build the model followed by using the model to predict a value:

```
from sklearn import tree
X = [[0, 0], [2, 2]]
y = [0.5, 2.5]
clf = tree.DecisionTreeRegressor()
clf = clf.fit(X, y)
clf.predict([[1, 1]])
```

Example 2.2.2 – Small Dataset without having to split the dataset

Type the following

```
#importing the libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

#importing the dataset
dataset=pd.read_csv('position_salaries.csv')

#Dataset's column begin with 0, 1,2
#dataset.iloc[:,1:2] gives you a 2-d dataframe (columns from 1 to 2)
#dataset.iloc[:,1] gives you a pandas series (1-d) (from column 1).
X=dataset.iloc[:,[1]].values
y=dataset.iloc[:,2].values
z=dataset.iloc[:,1].values
a=dataset.iloc[:,0].values

print(X)
print("\n")
print(y)
print("\n")
print(z)
print("\n")
print(a)

#fitting the decision tree regression model to the dataset
from sklearn.tree import DecisionTreeRegressor
regressor=DecisionTreeRegressor(random_state=0)
regressor.fit(X,y)
```

```
y_pred = regressor.predict([[6.5]])
print(y_pred)

#5 Visualising the Decision Tree Regression results
plt.scatter(X, y, color = 'red')
plt.plot(X, regressor.predict(X), color = 'blue')
plt.title('Check It (Regression Model)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```

Example 2.2.3 Load a big dataset and without split it into training as well as testing sets
Type the following and make sure you have downloaded the pima-diabetes.xlsx dataset.

```
#importing the libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

#importing the dataset
dataset=pd.read_excel('pima-indians-diabetes.xlsx')

#more explanation
https://pandas.pydata.org/pandas-#docs/stable/reference/api/pandas.DataFrame.iloc.html
#https://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-#
ix/
#Dataset's column begin with 0, 1,2
#dataset.iloc[:,[1]] gives you a 1-d dataframe (column 1)
#dataset.iloc[:,1] gives you a pandas series (1-d) (from column 1).
X=dataset.iloc[:,[1]].values
y=dataset.iloc[:,4].values


print(X)
print("\n")
print(y)
print("\n")

print(dataset['glucose'].count())
print(dataset['insulin'].count())
#print(y.count())


#fitting the decision tree regression model to the dataset without splitting the dataset
from sklearn.tree import DecisionTreeRegressor
regressor=DecisionTreeRegressor(random_state=0)
regressor.fit(X,y)
```

```
#5 Visualising the Decision Tree Regression results
plt.scatter(X, y, color = 'red')
plt.plot(X, regressor.predict(X), color = 'blue')
plt.title('Check It (Regression Model)')
plt.xlabel('Glucose level')
plt.ylabel('Insulin')
plt.show()
```

Example 2.2.4 Load a big dataset and without split it into training as well as testing sets
Type the following and make sure you have downloaded the pima-indians-diabetes.xlsx dataset.

```
#adapted from https://heartbeat.fritz.ai/implementing-regression-using-a-decision-tree-#and-scikit-learn-ac98552b43d7
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score,mean_squared_error
import seaborn as sns

#load the dataset
dataset = pd.read_excel('pima-indians-diabetes.xlsx')
dataset.head()

#split train and test dataset
X = dataset['glucose']
y = dataset['insulin']
X = X.values.reshape(-1, 1)
y = y.values.reshape(-1, 1)
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.50, random_state=42)

#Fit x_train and y-train into the regression model
#fitting the decision tree regression model to the dataset without splitting the dataset
from sklearn.tree import DecisionTreeRegressor
regressor=DecisionTreeRegressor(random_state=0)
regressor.fit(x_train,y_train)

#Obtain predicted y values (i.e. insulin) based on x test values
y_pred = regressor.predict(x_test)
print(y_pred)

#Model evaluation
#we evaluate our model by finding the root mean squared error produced by the model.
#use numpy np.sqrt
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("\n")
```

```
print("The root mean square error is: ", rmse)

#5 Visualising the Decision Tree Regression results based on x_train and y_train
plt.scatter(x_train, y_train, color = 'red')
plt.plot(x_test, regressor.predict(x_test), color = 'blue')
plt.title('Check It (Regression Model)')
plt.xlabel('Glucose level')
plt.ylabel('Insulin')
plt.show()
```

Linear Regression Notes:
https://scipy-lectures.org/packages/scikit-learn/index.html

Resources
https://scikit-learn.org/stable/modules/tree.html
https://heartbeat.fritz.ai/implementing-regression-using-a-decision-tree-and-scikit-learn-ac98552b43d7
https://scipy-lectures.org/packages/scikit-learn/index.html (KNN as well)

Resources
https://scikit-learn.org/stable/modules/tree.html
https://www.datacamp.com/community/tutorials/decision-tree-classification-python
https://machinelearningmastery.com/make-predictions-scikit-learn/

Additional Notes on how to use iloc to select rows and columns in pandas dataframe
https://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/
There are two "arguments" to iloc – a row selector, and a column selector. For example:

Iloc[rows, columns]

```
# Single selections using iloc and DataFrame
# Rows:
data.iloc[0] # first row of data frame (Aleshia Tomkiewicz) - Note a Series data type output.
data.iloc[1] # second row of data frame (Evan Zigomalas)
data.iloc[-1] # last row of data frame (Mi Richan)
# Columns:
data.iloc[:,0] # first column of data frame (first_name)
data.iloc[:,1] # second column of data frame (last_name)
data.iloc[:,-1] # last column of data frame (id)
```
view rawPandas Index - Single iloc selections.py hosted with ❤ by GitHub
Multiple columns and rows can be selected together using the .iloc indexer.

```
# Multiple row and column selections using iloc and DataFrame
```

```
data.iloc[0:5] # first five rows of dataframe
data.iloc[:, 0:2] # first two columns of data frame with all rows
data.iloc[[0,3,6,24], [0,5,6]] # 1st, 4th, 7th, 25th row + 1st 6th 7th columns.
data.iloc[0:5, 5:8] # first 5 rows and 5th, 6th, 7th columns of data frame (county -> phone1).
```